Engineering Team Handbook

Engineering Department - Infovance Technologies Pvt. Ltd.

Document ID: ENG-HB-2025-03	**Version:** 6.0	**Effective:** 01 April 2025
Owner: Engineering Department	**Classification:** Internal	**Next review:** 30 September 2025

1. About This Handbook

This handbook describes how the Engineering department at Infovance Technologies builds, reviews, ships and operates software. It is the shared reference for engineers across all squads and sets the expectations for collaboration, quality and delivery. New engineers should read it in full during their first week; existing engineers should treat it as the source of truth for our ways of working. Where a client engagement mandates different practices, the client agreement takes precedence and the deviation is documented in the project's Confluence space.

2. Team Structure

The Engineering department is organised into four disciplines that collaborate within cross-functional squads aligned to products and client programmes:

- **Frontend:** Builds responsive, accessible user interfaces and design-system components.
- **Backend:** Owns APIs, business logic, data models and service integrations.
- **DevOps / Platform:** Manages CI/CD, cloud infrastructure, observability and release engineering.
- **Quality Assurance (QA):** Owns test strategy, automation, and release quality gates.

Each squad is led by an Engineering Lead and supported by a Product Owner and a Scrum Master. Architectural decisions are reviewed by the Architecture Council, which maintains the technology radar and approves significant design changes.

3. Development Workflow and Git Branching

We use a trunk-aware Git Flow branching model. The protected long-lived branches are **main** and **develop**; short-lived branches are created for features and fixes.

Branch	Purpose	Rules
main	Production-ready code	Protected; only release/hotfix merges; tagged on release
develop	Integration of completed work	Protected; feature branches merge here via PR
feature/*	New features and changes	Branch from develop; e.g. feature/INFV-123-login
hotfix/*	Urgent production fixes	Branch from main; merged to main and back to develop

Branch names reference the Jira ticket key (for example, feature/INFV-451-citation-api). Commits follow the Conventional Commits style (feat:, fix:, chore:, refactor:, test:, docs:) to support automated changelog generation. Force-pushing to protected branches is prohibited.

4. Code Review Process

All changes reach protected branches through pull requests (PRs). A PR requires a **minimum of two approvals**, including at least one from a senior engineer or the discipline lead. Authors keep PRs small and focused, and provide a clear description with context, screenshots (for UI) and testing notes. Reviewers should respond within one working day. The reviewer checklist includes:

- Correctness: does the change do what it claims, including edge cases?

- Readability and maintainability; adherence to the team style guide and naming conventions.
- Adequate automated tests, with meaningful assertions.
- Security: input validation, no hard-coded secrets, safe handling of data.
- Performance and resource use are reasonable for the workload.
- No unintended scope creep; documentation and API contracts updated where relevant.

5. Sprint Ceremonies

Squads operate on two-week sprints using Scrum. The standard cadence is:

Ceremony	When	Purpose
Daily Stand-up	Every day, 10:00 AM (15 min)	Sync on progress, plan, and blockers
Sprint Planning	Monday (sprint start)	Commit to the sprint backlog and goals
Backlog Refinement	Mid-sprint	Clarify and estimate upcoming work
Sprint Review / Demo	Last day of sprint	Demo increment to stakeholders
Retrospective	Friday (sprint end)	Reflect and agree improvement actions

6. Technology Stack

Our default technology stack balances developer productivity, performance and long-term maintainability. Teams choose from the approved radar; new technologies require Architecture Council review.

- **Frontend:** React (with TypeScript), modern component libraries and a shared design system.
- **Backend:** Node.js services and Python services, exposing RESTful and event-driven APIs.
- **Data:** PostgreSQL as the primary relational store, with Redis for caching and queues.
- **Cloud:** Amazon Web Services (AWS) as the primary cloud, using managed services for compute, storage and networking.
- **Containers:** Docker images orchestrated on managed Kubernetes for production workloads.

7. Deployment Process

Continuous integration and delivery are powered by **Jenkins** pipelines. Every merge triggers automated build, test and security scans. The promotion path is strictly **staging before production**:

- On merge to develop, artefacts are built and deployed automatically to the **staging** environment.
- QA validates the release candidate in staging against the test plan and acceptance criteria.
- Production deployment requires a release approval and is performed during the agreed release window.
- Deployments use blue-green or rolling strategies to minimise downtime.
- **Rollback:** if a production issue is detected, the release is rolled back to the last known-good version by re-pointing traffic (blue-green) or re-deploying the previous artefact; a post-incident review follows.

8. On-Call Rotation

Production services are supported around the clock through an on-call rotation managed in **PagerDuty**. Engineers serve **two-week** on-call cycles, with a primary and a secondary responder. Alerts are routed by severity; the primary acknowledges within the agreed response time and engages the secondary or escalates if needed. Every page is logged, and significant incidents receive a blameless post-mortem with tracked action items. Compensatory off is provided for substantial out-of-hours engagement, in line with the Engineering on-call guidelines.

9. Documentation Standards

Good documentation is part of ‘done’. Our standards are:

- **Confluence** is the home for design docs, runbooks, architecture decision records (ADRs) and onboarding material.
- **Inline code comments** explain the ‘why’ behind non-obvious logic, not the obvious ‘what’.
- **API documentation** is generated and published via **Swagger / OpenAPI** and kept in sync with the code.
- Each service maintains a README covering setup, configuration and local-run instructions.
- Runbooks for on-call cover common alerts, diagnostics and recovery steps.

10. Definition of Done

A change is considered ‘done’ only when it meets a shared, non-negotiable bar of quality. Treating ‘done’ consistently is how we keep velocity honest and avoid hidden work piling up later. A work item is done when:

- Acceptance criteria in the Jira ticket are fully met and demonstrable.
- Code is peer-reviewed and merged through a pull request with the required approvals.
- Automated unit and integration tests are written and passing, and coverage gates are met.
- Relevant documentation, runbooks and API specs are updated.
- The change is deployed and verified in staging, with no open critical or high defects.
- Observability is in place (logs, metrics, alerts) for any new behaviour that warrants it.

If any of these are incomplete, the item stays in progress rather than being marked done. Carrying partially finished work as ‘done’ distorts velocity and pushes risk into the release window.

11. Performance KPIs and Quality Gates

We measure delivery health with a small set of meaningful indicators, used for improvement rather than individual ranking:

KPI	Target	Notes
Sprint velocity (story points)	Stable / predictable	Tracked per squad, not per person
Code coverage	> 80%	Enforced as a CI quality gate
Escaped defect rate	Low and trending down	Bugs found in production per release
PR cycle time	Short	Time from PR open to merge
Change failure rate	Low	Percentage of deployments causing incidents

Builds that fail to meet the coverage or security gates are blocked from promotion. Engineering leads review these metrics each sprint during the retrospective and agree concrete improvements. Questions about engineering practices can be raised in the #engineering channel or with your Engineering Lead.